

NAME: P.Surendranath reddy

REG NO: 192472356

COURSE CODE: CSA0606

COURSE NAME: Design and Analysis of Algorithms

## **Brute-Force Convex Hull Computation for Drone Obstacle Detection**

Q. A drone system identifies boundary points of obstacles using brute-force convex hull computation. (a) Explain how the convex hull is determined. (b) Analyze the time complexity. (c) Discuss why this approach is not scalable.

### **1. Objective**

To understand how a brute-force convex hull algorithm identifies the boundary points of obstacles for a drone system, analyze its computational complexity, and explain its scalability limitations.

### **2. Problem Interpretation**

A drone may detect an obstacle using a collection of points representing the obstacle boundary or surface. The convex hull is the smallest convex polygon that contains all the given points. The points that lie on this outer boundary are treated as the boundary points of the obstacle.

### **3. Brute-Force Convex Hull Approach**

The brute-force method examines every possible pair of points. For each pair, it considers the line joining them and checks all other points to determine whether they lie on the same side of the line. If all other points lie on one side, the pair can form an edge of the convex hull.

### **4. How the Convex Hull is Determined**

1. Take the set of detected obstacle points.
2. Select every possible pair of points as a candidate edge.
3. Draw the line through the selected pair.
4. Check the position of every other point relative to that line using an orientation/cross-product test.
5. If all other points are on the same side of the line, the selected points belong to the convex hull boundary.
6. Repeat the test for all point pairs.
7. Collect the boundary points/edges to obtain the convex hull.

### **5. Geometric Interpretation**

For two points P and Q, the algorithm checks whether every other point lies on the same side of line PQ. If no point occurs on both sides, PQ is a supporting edge of the convex hull. Repeating this process identifies the outermost boundary of the obstacle.

### **6. Example of Boundary Identification**

Suppose a drone detects points representing an obstacle. The algorithm tests pairs such as P1–P2, P1–P3, and so on. For each candidate pair, all remaining points are checked. Candidate edges for which all points lie on one side are retained. Connecting the retained boundary edges gives the convex hull.

### **7. Time Complexity Analysis**

Let  $n$  be the number of detected points. There are  $O(n^2)$  possible pairs of points. For each pair, the brute-force method checks up to  $O(n)$  other points. Therefore:

**Time Complexity =  $O(n^2) \times O(n) = O(n^3)$ .**

Thus, the brute-force convex hull approach has  **$O(n^3)$**  time complexity in the general case. This means that increasing the number of detected points significantly increases the amount of computation.

## 8. Space Complexity

The additional working space can be kept around  **$O(n)$**  for storing the detected points and the resulting boundary information, depending on the implementation.

## 9. Why the Approach is Not Scalable

- **Large number of comparisons:** Every pair of points is examined.
- **Repeated checking:** The same points are repeatedly tested against many candidate edges.
- **High computational cost:**  $O(n^3)$  growth becomes expensive as the number of sensor/detection points increases.
- **Real-time limitation:** A drone may need rapid obstacle detection and navigation decisions, but brute-force computation can introduce significant delay.
- **Sensor-density problem:** Higher-resolution sensors can produce many points, making the cubic growth particularly costly.

## 10. Complexity Summary

| Parameter               | Analysis                                                |
|-------------------------|---------------------------------------------------------|
| Input                   | n obstacle points                                       |
| Candidate point pairs   | $O(n^2)$                                                |
| Check for each pair     | $O(n)$                                                  |
| Overall time complexity | $O(n^3)$                                                |
| Additional space        | Approximately $O(n)$ , implementation-dependent         |
| Scalability             | Poor for large point sets / real-time dense sensor data |

## 11. Advantages

- Simple and straightforward concept.
- Easy to understand and implement.
- Directly checks candidate boundary edges.
- Useful for small point sets and educational purposes.

## 12. Limitations

- $O(n^3)$  time complexity makes it inefficient for large datasets.
- Repeated geometric tests waste computation.
- Not suitable for dense sensor data or high-frequency real-time obstacle detection.
- More efficient convex hull algorithms are preferable for practical large-scale systems.

## 13. Correctness Interpretation

The brute-force procedure correctly identifies an edge when all other points lie on the same side of its supporting line. The collection of such outer edges forms the convex hull, which encloses all detected obstacle points and represents their outer boundary.

## 14. Conclusion

Brute-force convex hull computation determines drone-obstacle boundary points by testing every pair of detected points and checking the position of all remaining points. The method has  **$O(n^3)$**  time complexity because  $O(n^2)$

candidate pairs each require up to  $O(n)$  point checks. Although simple and reliable for small datasets, its cubic growth makes it unsuitable for large point clouds and real-time drone applications. More efficient convex hull methods are needed when scalability is important.